

REFFI DWINO IGMAYODA (23)

254107060003

Sistem Informasi Bisnis

JOBSHEET 12

12.2 Percobaan 1: Operasi Penambahan pada Double Linked List

Doublelinklist23.java

```
public class Doublelinklist23 {
    Node23 head;
    Node23 tail;

    public Doublelinklist23() {
        head = null;
        tail = null;
    }

    public boolean isEmpty() {
        return head == null;
    }

    public void addFirst(Mahasiswa23 data) {
        Node23 newNode23 = new Node23(data);
        if (isEmpty()) {
            head = tail = newNode23;
        } else {
            newNode23.next = head;
            head.prev = newNode23;
            head = newNode23;
        }
    }

    public void addLast(Mahasiswa23 data) {
        Node23 newNode23 = new Node23(data);
        if (isEmpty()) {
            head = tail = newNode23;
        } else {
            tail.next = newNode23;
            newNode23.prev = tail;
            tail = newNode23;
        }
    }

    public void insertAfter(String keyNim, Mahasiswa23 data) {
        Node23 current = head;
        while (current != null && !current.data.nim.equals(keyNim)) {
            current = current.next;
        }
    }
}
```

DoublelinklistMain23.java

```
import java.util.Scanner;

public class DoublelinklistMain23 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        Doublelinklist23 list = new Doublelinklist23();
        int pilihan;

        do {
            System.out.println("\n==== MENU DOUBLE LINKED LIST ===");
            System.out.println("1. Tambah data di awal");
            System.out.println("2. Tambah data di akhir");
            System.out.println("3. Sisipkan data tengah (Setelah NIM)");
            System.out.println("4. Hapus data di awal");
            System.out.println("5. Hapus data di akhir");
            System.out.println("6. Tampilan data");
            System.out.println("0. Keluar");
            System.out.println("Pilih Menu :");

            pilihan = sc.nextInt();
            sc.nextLine();
            switch (pilihan) {
                case 1:
                    Mahasiswa23 mhsAwal = inputMahasiswa(sc);
                    list.addFirst(mhsAwal);
                    break;

                case 2:
                    Mahasiswa23 mhsAkhir = inputMahasiswa(sc);
                    list.addLast(mhsAkhir);

                case 3:
                    System.out.println("Masukkan NIM yang dicari :");
                    String keyNim = sc.nextLine();
                    System.out.println("Masukkan data baru :");
                    Mahasiswa23 databaru = inputMahasiswa(sc);
                    list.insertAfter(keyNim, databaru);
            }
        } while (pilihan != 0);
    }
}
```

Mahasiswa23.java

```
public class Mahasiswa23 {  
  
    String nim;  
    String nama;  
    String kelas;  
    double ipk;  
  
    public Mahasiswa23(String nim, String nama, String kelas, double ipk) {  
        this.nim = nim;  
        this.nama = nama;  
        this.kelas = kelas;  
        this.ipk = ipk;  
    }  
  
    public void tampil(){  
        System.out.println(  
            "NIM   :" + nim +  
            "\nNama  :" + nama +  
            "\nKelas:" + kelas +  
            "\nIPK   :" + ipk  
        );  
    }  
}
```

Node23.java

```
public class Node23 {  
    Mahasiswa23 data;  
    Node23 prev;  
    Node23 next;  
  
    public Node23(Mahasiswa23 data) {  
        this.data = data;  
        this.prev = null;  
        this.next = null;  
    }  
}
```

Pertanyaan

1. Jelaskan perbedaan struktur dan mekanisme traversal antara Single Linked List dan Double Linked List!

Aspek	Single Linked List	Double Linked List
Struktur Node	Setiap node hanya memiliki satu pointer/referensi ke node berikutnya (next).	Setiap node memiliki dua pointer: next (ke node berikutnya) dan prev (ke node sebelumnya).
Arah Traversal	Hanya bisa maju (dari head menuju tail) karena tidak ada referensi ke node sebelumnya.	Bisa maju (dari head ke tail) dan mundur (dari tail ke head) karena memiliki prev.
Efisiensi Operasi	Operasi seperti hapus node tengah memerlukan traversal dari head untuk mendapatkan node sebelumnya, kompleksitas $O(n)$.	Operasi hapus node tengah lebih mudah karena langsung mengakses prev dari node yang akan dihapus, tanpa perlu traversal ulang.
Penggunaan Memori	Lebih hemat memori (hanya satu pointer per node).	Memerlukan memori lebih besar (dua pointer per node).
Penanganan Tail	Menghapus node terakhir tetap perlu traversal untuk menemukan node sebelum tail jika tidak disimpan.	Dengan atribut tail, penghapusan node terakhir bisa langsung menggunakan tail.prev, lebih cepat.

2. Perhatikan class Node, di dalamnya terdapat atribut next dan prev. Jelaskan fungsi masing-masing atribut tersebut pada proses traversal dan manipulasi node!
 - next: Menyimpan referensi ke node berikutnya. Digunakan untuk traversal maju (dari head ke tail) dan menghubungkan node saat operasi penyisipan/penghapusan.
 - prev: Menyimpan referensi ke node sebelumnya. Memungkinkan traversal mundur (dari tail ke head) serta mempermudah operasi seperti penghapusan node tengah (tanpa perlu mencari node sebelumnya dari awal) dan penyisipan sebelum suatu node.

3. Perhatikan konstruktor pada class DoubleLinkedList. Jelaskan fungsi konstruktor tersebut terhadap kondisi awal linked list!

Fungsi konstruktor DoubleLinkedList23()

Menginisialisasi linked list dalam keadaan kosong dengan head = null dan tail = null.

4. Perhatikan potongan kode berikut: if (isEmpty()) { head = tail = newNode; } Mengapa head dan tail harus menunjuk node yang sama ketika linked list masih kosong?
Karena node pertama sekaligus menjadi node pertama dan terakhir, sehingga kedua pointer harus menunjuk ke node yang sama untuk menjaga konsistensi operasi.
5. Modifikasi method print() agar menampilkan pesan "Linked List masih kosong" ketika tidak terdapat data pada linked list!

```
public void print() {  
    if (isEmpty()) {  
        System.out.println("Linked List masih kosong");  
        return;  
    }  
    System.out.println("=== Data Mahasiswa ===");  
    Node23 current = head;  
    while (current != null) {  
        current.data.tampil();  
        System.out.println();  
        current = current.next;  
    }  
}
```

6. Modifikasi kode program dengan menambahkan method printReverse() untuk menampilkan seluruh data pada Double Linked List secara terbalik, dimulai dari node tail menuju head!

```
public void printReverse() {  
    if (isEmpty()) {  
        System.out.println("Linked List masih kosong");  
        return;  
    }  
    System.out.println("=== Data Mahasiswa (dari belakang) ===");  
    Node23 current = tail;  
    while (current != null) {  
        current.data.tampil();  
        System.out.println();  
        current = current.prev;  
    }  
}
```

Hasilnyaa

```
=== MENU DOUBLE LINKED LIST ===
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data tengah (Setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih Menu : 6
NIM :31231
Nama :wqeq
Kelas :123esda
IPK :21.0
```

12.3 Percobaan 2: Operasi Penghapusan pada Double Linked List

```
public void removeLast() {
    if (!isEmpty()) {
        System.out.println(x: "Linked list kosong ");
        return;
    }

    if (head == tail) {
        head = tail = null;
    } else {
        tail = tail.prev;
        tail.next = null;
    }
}

public void printReverse() {
    if (isEmpty()) {
        System.out.println(x: "Linked List masih kosong");
        return;
    }
    Node23 current = tail;
    while (current != null) {
        current.data.tampil();
        current = current.prev;
    }
}
```

Hasilnya

```
=== MENU DOUBLE LINKED LIST ===
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data tengah (Setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih Menu : 3
Masukkan NIM yang dicari : 31231
Masukkan data baru :
Masukkan NIM : 31231
Masukkan Nama : wqeq
Masukkan Kelas : 123esda
Masukkan IPK : 21
Data berhasil disisipkan setelah NIM 31231

=== MENU DOUBLE LINKED LIST ===
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data tengah (Setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih Menu : 
```